

PROJECT 2 — INVESTMENT DATA WAREHOUSE & ML

Value Investing Challenge · RA2

EVERY PREDICTION MUST BE RECORDED BEFORE WE KNOW THE OUTCOME.

Una predicción no es "esta acción va a subir". Una predicción es: *"dada la información disponible hoy, mi sistema estima un 67% de probabilidad de que esta acción alcance al menos +10% en los próximos 3 meses"*. En RA2 esa frase deja de ser una intuición ponderada a mano y pasa a ser la salida de un modelo entrenado. Lo cual introduce un peligro nuevo: un modelo puede aprender del futuro sin que os deis cuenta.

1. Introducción del proyecto

En RA1 construisteis un Data Lake y un scoring manual (`v1`). Ahora transformáis ese lake en una plataforma analítica: orquestación con Airflow, un Data Warehouse dimensional en PostgreSQL, tablas de features con corrección point-in-time y modelos de Machine Learning que estiman `P(Return >= 10%)` por horizonte.

Seguís sin construir la web. La plataforma **Value Investing Challenge** consulta vuestro endpoint cada ciclo semanal, congela la respuesta y os rankea. La tarea de predicción es idéntica: **Top 3 tickers por horizonte** para `1W` , `1M` , `3M` , `6M` , **máximo 12 predicciones por ciclo**. Una predicción es **hit** cuando `realized_return >= 0.10` .

En RA2 vuestro modelo se registra como `v2` . Los resultados de `v1` no se sobrescriben. Al final de RA2 tendréis por primera vez la respuesta empírica a la primera gran pregunta del curso: **¿el ML batió al scoring manual?** Es perfectamente posible que no. Con tasas base de 9% en 1W y 10 tickers en el universo, un modelo mal validado pierde contra una fórmula sensata. Detectarlo y explicar por qué es el aprendizaje; ocultarlo no.

2. Objetivos de aprendizaje

- Orquestar un pipeline de datos con Apache Airflow: dependencias, idempotencia, reintentos, backfill.
- Diseñar un modelo dimensional (hechos y dimensiones) para datos financieros y justificar la granularidad.
- Implementar controles de calidad como tareas del DAG que puedan fallar el pipeline.

- Construir `STOCK_FEATURES` con **corrección point-in-time**, entendiendo cómo se cuela el look-ahead bias.
- Entrenar y **comparar** modelos de clasificación probabilística por horizonte.
- Evaluar con métricas adecuadas a clases desbalanceadas: PR-AUC, Brier, calibración.
- Calibrar probabilidades con Platt scaling o isotonic regression.
- Validar cronológicamente con walk-forward, no con splits aleatorios.

3. Arquitectura



4. Requisitos

Obligatorio: un DAG de Airflow funcional, controles de calidad de datos, un modelo dimensional con hechos y dimensiones, feature engineering, snapshots históricos, y al menos dos modelos de ML comparados entre sí.

Modelo dimensional propuesto

Podéis adaptarlo, pero debéis justificar los cambios:

```

DIM_COMPANY, DIM_DATE, DIM_SECTOR
FACT_MARKET_DATA, FACT_FINANCIALS, FACT_EARNINGS,
FACT_NEWS_SIGNALS, FACT_COMPETITOR_METRICS, FACT_PREDICTIONS
  
```

Decisiones que debéis tomar explícitamente:

DECISIÓN	OPCIONES	CONSECUENCIA
Granularidad de <code>FACT_MARKET_DATA</code>	Diaria vs intradía	Diaria basta para horizontes de 1W a 6M

DECISIÓN	OPCIONES	CONSECUENCIA
Historia en <code>DIM_COMPANY</code>	SCD tipo 1 vs tipo 2	Tipo 2 permite saber en qué sector estaba una empresa en el pasado
Clave de <code>FACT_FINANCIALS</code>	<code>fiscal_period_end</code> vs <code>published_at</code>	Determina si tenéis look-ahead bias o no
Restatements	Sobreescribir vs versionar	Versionar es la única opción compatible con point-in-time

`FACT_PREDICTIONS` guarda vuestras propias predicciones con su `generated_at`. Es vuestro registro local paralelo al de la plataforma y os permite auditar `v1` vs `v2` vs `v3` con vuestros propios datos.

STOCK_FEATURES: la tabla más importante

Una fila = un ticker en una fecha, construida **exclusivamente con información disponible hasta esa fecha**.

Lista realista de columnas: `ticker`, `snapshot_date`, `pe_ratio`, `forward_pe`, `ev_ebitda`, `price_to_sales`, `fcf_yield`, `revenue_growth_yoy`, `eps_growth_yoy`, `ebitda_growth_yoy`, `gross_margin`, `operating_margin`, `net_margin`, `roe`, `roa`, `roic`, `debt_to_equity`, `net_debt_ebitda`, `current_ratio`, `momentum_1w`, `momentum_1m`, `momentum_3m`, `momentum_6m`, `volatility_30d`, `volatility_90d`, `volume_change`, `earnings_surprise`, `earnings_sentiment`, `sector_relative_pe`, `sector_relative_growth`, `sector_relative_roic`, `analyst_signal`, `news_sentiment`.

No estáis obligados a tener todas. Decidid qué podéis obtener de forma fiable y qué aporta valor. Veinte features bien construidas y point-in-time correctas valen más que treinta y tres con la mitad imputadas a ojo.

5. Point-in-time correctness

Esta sección es el núcleo del RA2. Un modelo que ve el futuro obtiene métricas excelentes en validación y falla en producción. Y en este curso "producción" es el leaderboard.

Cómo se cuela el look-ahead bias

1. Usar un fundamental en la fecha de cierre del trimestre fiscal en lugar de la fecha de publicación. Es el error más frecuente. Un trimestre cerrado el 30 de septiembre se publica típicamente entre 30 y 45 días más tarde. Si unís por `fiscal_period_end`, vuestro modelo conoce el beneficio del Q3 a finales de septiembre, cuando el mercado no lo conocerá hasta noviembre.

Arreglo: unir siempre por `published_at <= snapshot_date` y quedarse con la última publicación disponible.

```

-- INCORRECTO: el modelo ve el trimestre antes de que se publique
SELECT f.ticker, d.snapshot_date, f.roic
FROM snapshots d
JOIN fact_financials f
  ON f.ticker = d.ticker
 AND f.fiscal_period_end <= d.snapshot_date;

-- CORRECTO: solo lo que estaba publicado en snapshot_date
SELECT s.ticker, s.snapshot_date, f.roic, f.published_at
FROM snapshots s
JOIN LATERAL (
  SELECT roic, published_at
  FROM fact_financials f
  WHERE f.ticker = s.ticker
        AND f.published_at <= s.snapshot_date
  ORDER BY f.published_at DESC
  LIMIT 1
) f ON TRUE;

```

2. Usar una cifra reexpresada (restated). Las empresas revisan cifras. Si vuestra tabla guarda el valor final corregido, estáis usando información que no existía en la fecha del snapshot. **Arreglo:** versionar cada hecho con `published_at` y `version`, y filtrar por la versión vigente en la fecha.

3. Normalizar sobre todo el dataset, incluido el futuro. Un z-score o min-max calculado con la media y desviación de toda la serie inyecta información del futuro en cada fila. **Arreglo:** calcular la normalización cross-sectional dentro de cada `snapshot_date` (que además es lo natural para un ranking), o con ventana expanding que solo mire hacia atrás.

4. Split train/test aleatorio. Con series temporales, un split aleatorio pone el martes en test y el miércoles en train. El modelo interpola en lugar de extrapolar y el resultado es fantasía. **Arreglo:** split cronológico estricto.

5. Survivorship bias. Usar la composición del índice de hoy para fechas pasadas elimina a las empresas que quebraron o fueron excluidas. El universo resultante solo contiene supervivientes y cualquier estrategia parece rentable. **Arreglo:** usar composición histórica; si no la tenéis, decir explícitamente en el informe que vuestro universo fijo de 10 tickers tiene este sesgo y qué implica.

6. Data leakage por la variable objetivo. Si `momentum_1w` en el snapshot se calcula con el precio de cierre del día siguiente, o si el target y una feature comparten el mismo periodo de precios, hay fuga directa. **Arreglo:** que toda feature use datos con fecha `< snapshot_date` y que el target use exclusivamente el intervalo `(snapshot_date, snapshot_date + horizonte]`.

Test de humo

Reconstruid `STOCK_FEATURES` para un `snapshot_date` de hace seis meses usando solo datos con `published_at` anterior a esa fecha. Si alguna columna cambia respecto a la que teníais almacenada, tenéis look-ahead bias. Este test debería ser una tarea del DAG.

6. Tecnologías recomendadas

Apache Airflow, SQL, PostgreSQL (**NeonDB o Supabase**, cualquiera de las dos: ambas son Postgres con free tier, no se impone ninguna), Python, scikit-learn, y opcionalmente XGBoost o LightGBM.

COMPONENTE	OPCIÓN	NOTA REAL
Postgres gestionado	NeonDB	Branching de base de datos, escala a cero (primera query tras inactividad es lenta)
Postgres gestionado	Supabase	Postgres con API REST y auth incluidas; el free tier pausa proyectos inactivos
Airflow	Local con Docker Compose	Lo más barato y suficiente; consume RAM
Airflow	Astronomer / Composer free credits	Menos setup, con caducidad de créditos

Si el free tier pausa vuestra base de datos y el ciclo semanal cae en ese momento, no hay predicciones. Contad con ello.

Airflow: qué se espera del DAG

- Tareas separadas: extracción desde el lake, limpieza, carga de dimensiones, carga de hechos, construcción de features, validación de calidad, entrenamiento/inferencia, publicación.
- **Idempotencia**: reejecutar una fecha debe producir el mismo resultado, no duplicar filas. Usad `MERGE / INSERT ... ON CONFLICT`.
- Dependencias explícitas: los hechos no se cargan antes que las dimensiones.
- Los checks de calidad deben poder **fallar la tarea**. Un check que solo hace log no es un check.
- Reintentos con backoff para tareas que dependen de APIs externas.
- Backfill funcional: debéis poder reconstruir el histórico de snapshots.

7. Machine Learning

Target

`P(Return >= 10%)` dentro del horizonte, por horizonte. Un modelo por horizonte es preferible a un modelo único con `horizon` como feature, por tres razones concretas: las tasas base son muy distintas (9% vs 42%), las features relevantes cambian con el horizonte (momentum manda en 1W, fundamentales en 6M), y un modelo único con la clase mayoritaria dominada por 1W tiende a colapsar a "no" en todos los horizontes. Cuatro modelos también significan cuatro veces menos datos cada uno: si vuestro histórico es corto, decidlo y justificad el compromiso.

Algoritmos candidatos

Logistic Regression, Random Forest, Gradient Boosting, XGBoost, LightGBM, una red neuronal pequeña, ensembles. **Debéis comparar modelos, no usarlos todos.** Dos o tres bien evaluados, con la comparación en una tabla, valen más que siete entrenados sin criterio. Empezad por Logistic Regression como baseline: si un XGBoost con 40 hiperparámetros no bate a una regresión logística, el problema no es el modelo.

Evaluación

Accuracy, Precision, Recall, F1, ROC-AUC, PR-AUC, Brier, Calibration.

Con una tasa base de 9% en 1W, la accuracy es inútil: un modelo que siempre dice "no" acierta el 91% y no sirve para nada. Lo que importa es **PR-AUC, Brier y calibración**. PR-AUC se centra en la clase positiva, que es la que os interesa. Brier mide error probabilístico. La calibración mide si vuestro 0.30 significa realmente 30%.

Calibración de probabilidades

Los modelos de gradient boosting optimizan log-loss o similar pero sus salidas no son probabilidades bien calibradas: un XGBoost sin calibrar produce probabilidades con aspecto seguro (0.85, 0.90) que destrozan el Brier score cuando la tasa base real es 9%. Random Forest, por promediar votos, tiende a comprimir hacia el centro.

Dos técnicas, ambas en `sklearn.calibration.CalibratedClassifierCV`:

- **Platt scaling** (`method="sigmoid"`): ajusta una regresión logística sobre las puntuaciones del modelo. Pocos parámetros, funciona con pocos datos. Asume forma sigmoide.
- **Isotonic regression** (`method="isotonic"`): ajusta una función monótona libre. Más flexible, pero **sobreajusta con muestras pequeñas**, que es exactamente vuestro caso en 1W.

Regla práctica: con pocos positivos, Platt. Calibrad siempre sobre un conjunto **separado** del de entrenamiento, y cronológicamente posterior. Reportad Brier antes y después de calibrar: es una de las tablas más informativas que podéis entregar.

Validación cronológica y walk-forward

Split cronológico obligatorio: train hasta `T1`, validación de `T1` a `T2`, test de `T2` en adelante. Nunca aleatorio.

Walk-forward: en lugar de un único corte, entrenad con la ventana disponible, predecid el periodo siguiente, avanzad la ventana, reentrenad y repetid. Obtenéis varias evaluaciones fuera de muestra en lugar de una, y veis si el rendimiento es estable o dependía de un periodo concreto.

```
Fold 1: train [2024-01 .. 2024-09] test [2024-10 .. 2024-12]
Fold 2: train [2024-01 .. 2024-12] test [2025-01 .. 2025-03]
Fold 3: train [2024-01 .. 2025-03] test [2025-04 .. 2025-06]
```

Respetad el **embargo**: si el horizonte es 6M, una predicción hecha en `T` no se resuelve hasta `T+6M`, así que test no puede empezar antes de que las etiquetas de train estén cerradas. Sin ese hueco hay solapamiento de etiquetas, que es fuga temporal.

Overfitting

Con 10 tickers y un histórico corto, el sobreajuste es el escenario por defecto. Señales: métricas de train muy superiores a las de test, resultados que se desmoronan al cambiar de fold, importancia de features que cambia radicalmente entre folds. Contramedidas: regularización, profundidad limitada, menos features, y validación walk-forward en lugar de un único split afortunado.

8. Requisitos mínimos (para aprobar)

- DAG de Airflow que se ejecuta de principio a fin sin intervención manual.
- Data Warehouse en PostgreSQL con al menos 3 dimensiones y 3 tablas de hechos poblados.
- `STOCK_FEATURES` con snapshots históricos y join point-in-time demostrable en SQL.
- Al menos 3 checks de calidad implementados como tareas que pueden fallar el DAG.
- Al menos dos modelos comparados por horizonte, con tabla de métricas incluyendo PR-AUC y Brier.
- Split cronológico y probabilidades calibradas.
- Endpoint `GET /api/predictions` sirviendo `model_version: "v2"` con payload válido.
- Comparación documentada de `v2` frente a vuestro propio `v1`.

9. Opcional / bonus

- Walk-forward completo con métricas por fold y su dispersión.
- SCD tipo 2 en `DIM_COMPANY`.
- Feature store con versionado y linaje.
- Registro de experimentos (MLflow o equivalente) con hiperparámetros y métricas.
- Explicabilidad con SHAP y contraste con los pesos que elegisteis a mano en RA1.
- Tests de data quality declarativos (Great Expectations, dbt tests).
- Análisis de la degradación del modelo entre ciclos.

10. Entregables

1. Repositorio con el DAG, las transformaciones SQL y el código de ML.
2. Esquema del Data Warehouse (DDL real y diagrama).
3. Documentación del join point-in-time con el SQL que lo implementa.

4. Informe de calidad de datos generado por el pipeline.
5. Notebook de entrenamiento y evaluación con la tabla comparativa de modelos.
6. Curvas de calibración por horizonte, antes y después de calibrar.
7. Endpoint desplegado sirviendo `v2`.
8. **Informe `v1` vs `v2`** : hit rate, Brier, Brier Skill Score y alpha de ambas versiones, con interpretación honesta.

11. Rúbrica de evaluación

CRITERIO	PESO	INSUFICIENTE	CORRECTO	EXCELENTE
Orquestación con Airflow	15%	Scripts manuales sin DAG	DAG con dependencias y reintentos	Idempotente, con backfill probado y checks que fallan el pipeline
Modelo dimensional	15%	Tablas planas sin diseño	Hechos y dimensiones con granularidad justificada	SCD tipo 2, claves e índices razonados, DDL versionado
Point-in-time correctness	20%	Join por <code>fiscal_period_end</code>	Join por <code>published_at <= snapshot_date</code>	Hechos versionados, normalización sin fuga, test de reconstrucción automatizado
Feature engineering	15%	Copia directa de campos crudos	Features derivadas y relativas al sector	Features justificadas por hipótesis económica y validadas contra fuga
Modelado y comparación	15%	Un solo modelo sin baseline	Dos o más modelos comparados con métricas adecuadas	Walk-forward, análisis de estabilidad entre folds, explicabilidad
Calibración y evaluación	10%	Solo accuracy	PR-AUC y Brier reportados	Calibración aplicada y medida, Brier antes/después, conciencia de las tasas base
Integración y análisis v1 vs v2	10%	Sin comparación	Comparación numérica de ambas versiones	Análisis causal de por qué ganó o perdió el ML, like-for-like

12. Errores comunes

- **Unir fundamentales por fecha de cierre fiscal.** Look-ahead bias de 30 a 45 días.
- **Normalizar sobre todo el dataset.** Fuga silenciosa que no aparece en ninguna métrica.
- `train_test_split(shuffle=True)` en datos temporales.

- Solapar etiquetas entre train y test sin embargo temporal.
- Reportar accuracy en 1W y creer que un 91% es bueno.
- XGBoost sin calibrar. Buen ranking, Brier desastroso.
- Isotonic regression con 30 positivos. Sobreajusta la calibración misma.
- DAG no idempotente que duplica filas en cada reejecución.
- Checks de calidad que solo hacen log y nunca detienen el pipeline.
- Base de datos pausada por inactividad el día de la recogida.
- Cambiar de universo o de horizontes entre v1 y v2 y luego comparar. La comparación debe ser like-for-like.
- Entrenar con el dataset sintético de data/mock/ y enviar esas predicciones.

13. El dataset de referencia

En data/mock/ del repositorio de la plataforma, en CSV y JSON: companies.csv, market_data.csv, fundamentals.csv, earnings.csv, competitors.csv, news_signals.csv, macro.csv, stock_features.csv, predictions.csv, prediction_results.csv, cycles.csv, students.csv.

Universo: AAPL, MSFT, NVDA, META, GOOGL, AMZN, TSLA, JPM, V, NFLX. Sintético pero internamente consistente: un mayor revenue_growth produce realmente un growth_score mayor. Sirve para desarrollar el DAG y probar el esquema antes de tener datos propios. stock_features.csv es una referencia útil de estructura para vuestra propia tabla.

No enviéis predicciones basadas en este dataset sintético. Es solo para desarrollo.

El baseline de referencia del dataset usa:

```
baseline_score = 0.20*valuation + 0.20*growth + 0.20*quality
                + 0.15*financial_health + 0.15*momentum + 0.10*earnings
```

Es el punto de comparación natural: si vuestro ML no bate a esta fórmula de seis términos, hay algo que explicar.

14. Cómo se os puntúa

Métricas por alumno: Resolved predictions, Hit Rate, Average/Median/Best/Worst Return, Average Alpha (contra un benchmark equiponderado), Average Probability, Brier Score, Brier Skill Score, Calibration y Consistency.

Brier score: media de $(p - y)^2$ con $y=1$ si hit. Más bajo, mejor. Un modelo que declara siempre la tasa base obtiene el Brier de referencia; batirlo exige información real.

Brier Skill Score: $1 - \text{brier} / \text{brier_reference}$, donde la referencia siempre es la tasa base de la cohorte para ese horizonte. Positivo significa que vuestras probabilidades aportan información. **Negativo significa que son activamente engañosas**, y ahí es donde aterriza el exceso de confianza.

Calibration: las predicciones se agrupan en 10 bins de probabilidad y se compara la probabilidad declarada con la tasa de éxito observada. Si en el bin 0.6–0.7 acertáis el 20% de las veces, estáis descalibrados.

Leaderboard score (0–100) = 30% hit rate + 25% calibration + 25% relative return + 10% consistency + 10% sample reliability. Cada componente se mide **relativo a la cohorte**, no contra una constante absoluta: si la semana fue mala para todos, no os penaliza en términos relativos.

Se exigen **mínimo 12 predicciones resueltas** para aparecer en el ranking principal.

Las tasas base, otra vez

HORIZONTE	HIT RATE OBSERVADO EN EL DATASET DE REFERENCIA
1W	≈ 9%
1M	≈ 35%
3M	≈ 43%
6M	≈ 42%

Alcanzar +10% es mucho más difícil en horizontes cortos. **Quien declare 70% de probabilidad en un pick de 1W tiene garantizado un Brier score terrible.** En RA2 esto tiene una consecuencia técnica directa: si vuestro modelo de 1W emite probabilidades medias de 0.5, está descalibrado por construcción y la calibración (25% del leaderboard score) lo va a reflejar.

15. Formato de entrega

- Repositorio Git con historial de commits.
- README con: cómo levantar Airflow, cómo aplicar el DDL, variables de entorno necesarias (sin credenciales en el repositorio), cómo entrenar y cómo servir el endpoint.
- DDL del warehouse versionado en el repositorio.
- Dependencias declaradas de forma reproducible.
- Endpoint accesible por HTTPS y respondiendo durante el ciclo de recogida.
- Informe `v1` vs `v2` en Markdown dentro del repositorio.

16. Conexión con el Value Investing Challenge

La plataforma consulta vuestro endpoint una vez por ciclo, guarda la respuesta de forma inmutable y no la modifica jamás. Cuando el horizonte vence, calcula el retorno realizado y resuelve la

predicción. Toda predicción queda registrada antes de conocer el resultado.

Contrato del alumno

GET /api/predictions debe devolver:

```
{
  "student_id": "student-01",
  "model_version": "v1",
  "model_name": "Weighted Value Score",
  "generated_at": "2026-09-14T12:00:00Z",
  "predictions": [
    {
      "ticker": "META",
      "horizon": "1W",
      "rank": 1,
      "probability": 0.71,
      "expected_return": 0.14,
      "target_price": 712.40,
      "investment_thesis": "optional free text",
      "risks": "optional free text"
    }
  ]
}
```

Reglas de validación (Zod) aplicadas por la plataforma: `horizon` $\in \{1W, 1M, 3M, 6M\}$; `rank` $\in 1..3$ y único por horizonte; `probability` $\in [0, 1]$; `expected_return` $\in [-1, 10]$; `generated_at` debe ser un datetime ISO-8601 **con offset**; ticker en mayúsculas; sin tickers duplicados dentro del mismo horizonte; máximo 12 predicciones. **Un payload que falla la validación se rechaza por completo y esa semana el alumno no puntúa.** La guía completa está en `docs/STUDENT_INTEGRATION.md`.

En RA2 debéis enviar `model_version: "v2"`. El ejemplo de arriba muestra `v1` porque es el contrato literal; cambiad ese campo.

La plataforma conserva `v1` y `v2` por separado y los compara. Al terminar RA2 debéis poder responder con números: **¿el ML batió al scoring manual?** Si la respuesta es no, el trabajo consiste en explicar por qué —muestra pequeña, features débiles, calibración pobre, universo demasiado estrecho— y esa explicación puntúa. Lo que no puntúa es presentar solo el horizonte o la semana en la que `v2` quedó por delante.

Every prediction must be recorded before we know the outcome. • PROJECT 2 — INVESTMENT DATA WAREHOUSE & ML